

DD: PLM-CFG-03 — Wallet Configuration

Developer implementation guide: DD_PLM-CFG-03-Wallet_Configuration-DEV_IMPL_GUIDE-v1.0.md.

1. Purpose

This rewritten DD is the developer-facing version of the module contract
DD_PLM-CFG-03-Wallet_Configuration-v1.0.md.

Verdict: ■ New | **Wave:** 4 | **Group VII:** Configuration

The goal of this rewrite is to make the DD easier for a mid-level developer to implement without losing the detailed source contract.

2. Implementation Scope

This DD should be implemented as part of the owning SOPHIX capability, not as an isolated service unless the deployment architecture explicitly separates that capability.

Implement this DD with these rules:

- Use the owning module APIs/repositories for authoritative writes.
- Use approved internal APIs/client wrappers when reading another module's data.
- Do not query another module's database tables directly from business flow code.
- Treat worker names as logical Camunda external-task topics, not separate deployments.
- One worker application may handle multiple topics, but metrics, retries, and logs must remain visible per topic.
- Emit success events only after the authoritative state commit succeeds.
- Use outbox publishing for Kafka events.

3. Runtime Metadata

Item	Value
Source file	/Users/macbook/Downloads/Sophix_V3_BSS_DDs_MD_2026-05-27/09_catalogs/DD_PLM-CFG-03-Wallet_Configuration-v1.0.md
Version	v1.0
DD type	module contract
BPMN/process keys	Not explicitly defined
Drools packages	Not explicitly defined

4. Runtime Contracts To Implement

4.1 APIs

- DELETE /api/wallet-types/{id}
- DELETE /api/wallets/{id}
- GET /api/wallet-types
- GET /api/wallet-types/by-code/{code}
- GET /api/wallet-types/{id}
- GET /api/wallets
- GET /api/wallets/by-code/{code}
- GET /api/wallets/{id}
- PATCH /api/wallet-types/{id}
- PATCH /api/wallets/{id}

- POST /api/wallet-types
- POST /api/wallet-types/{id}/activate
- POST /api/wallet-types/{id}/retire
- POST /api/wallets
- POST /api/wallets/{id}/activate
- POST /api/wallets/{id}/retire

4.2 Tables

- No CREATE TABLE block is explicitly declared in this DD.

For each table in this DD, implement the schema with:

- a short table comment explaining what the table is for;
- column comments or migration notes explaining each field;
- constraints and indexes from the DD;
- seed data where the DD provides operator-specific sample rows.

4.3 Worker Topics

- plm-rules

Worker-topic guidance:

- A BPMN service task uses the topic.
- A worker application subscribes to the topic.
- The topic handler validates input variables, calls the owning module/API, writes outputs back to Camunda, and records metrics.
- Do not treat the topic string as a shell command or Kubernetes deployment name.

4.4 Events

- PaymentResource

Event guidance:

- Write events through the transactional outbox.
- Include operation id, aggregate id, operator code, actor, and correlation data.
- Consumers should be able to rebuild downstream state without querying workflow internals.

5. Developer Implementation Checklist

1. Add or verify database migrations and seed rows.
2. Add or verify config rows for the operator/module.
3. Implement request/command DTOs and validation.
4. Implement idempotency and in-flight operation checks where the DD requires them.
5. Implement Drools fact mapping and package deployment where rules are used.
6. Implement BPMN process, service tasks, gateways, timers, and message catches where the DD is a flow.
7. Implement worker-topic handlers using owning module APIs.
8. Implement compensation paths and manual recovery paths.
9. Emit success/rejected events through the outbox.
10. Add smoke tests for happy path, validation failure, dependency failure, and retry/idempotency.

6. Infrastructure And AWS Notes

Deploy this DD with the existing SOPHIX platform components unless the owning module requires isolation:

Concern	Recommendation
API/runtime	Run inside the owning module deployment or existing workflow API pods.
Workers	Consolidate low-volume topics into shared worker apps; keep per-topic metrics.
Database	Use the owning module PostgreSQL schema and migration pipeline.
Kafka	Use the foundation Kafka/MSK setup and transactional outbox.
Camunda	Deploy BPMN files through the workflow deployment pipeline.
Drools	Deploy KJAR/rule artifacts through the approved rules pipeline.
Secrets	Use the platform secret manager; on AWS prefer Secrets Manager/Parameter Store with IRSA.
Observability	Alert on stuck operations, failed workers, outbox backlog, and external dependency failures.

7. Original DD Detail Preserved

The following section preserves the detailed source contract for implementation and review.

Verdict: ■ New | **Wave:** 4 | **Group VII:** Configuration

A — Target

1. What this process is about

A **Wallet** is a deployment-configurable category of monetary-equivalent value held by the customer. Every wallet's balance is denominated in a currency (or in points convertible to a currency at a configured rate) and is used at charging time to settle the customer's obligations. The catalog tells downstream modules what wallets EXIST, how they BEHAVE (refill, expiry, negative-balance allowance, charging precedence, points-to-currency conversion where applicable), and what they REPRESENT in finance terms. The actual per-customer balances (the ledger) live in a separate billing sub-module (future BIL-06 / WAL-OP), out of scope here.

The value held in a wallet originates from one of several **sources**, each with distinct finance and operational semantics:

- **Customer top-up** — money the customer paid in (cash, card, mobile money, bank transfer). The "real money" balance.
- **Operator-granted bonus** — money the operator gifted as a goodwill or retention credit. Spent like money but tracked separately for finance reconciliation.
- **Promotional credit** — one-shot, time-limited grants (welcome bonus, referral reward, campaign credit). Typically charged against first because they expire.
- **Refundable deposit** — money held at install against contract termination or equipment damage. Cannot be spent by the customer; only the operator moves funds in or out. Finance treats this as a balance-sheet liability.
- **Loyalty points** — earned through activity (longevity, payments, referrals), stored as points internally and converted to currency at charging time via a per-wallet conversion rate.
- **SLA-driven service credit** — operator-issued compensation for service-level breaches. Tracked separately to demonstrate SLA compliance and to keep it out of normal bonus/promo accounting.
- **Hardware buyback credit** — issued when a customer returns equipment in good condition at contract end. Redeemable against future purchases or refundable.

The wallet does NOT hold data MB, voice minutes, or SMS counts. Those are bucket-style allowances, modeled separately (deferred to a future DD). The model here is intentionally money-only — one numeric balance per (customer, wallet) used at charging time.

The wallet is the unit of value the charging path uses. When a Service or charge runs, fulfillment / billing picks the right wallet from the catalog by reference (`walletRef`), reads the customer's current balance from BIL-06's ledger, and applies the rules defined in PLM-CFG-03 (can balance go negative, precedence among multiple eligible wallets, expiry behavior, refill-from rules). The catalog is therefore the source of truth for "what wallets does the deployment offer and how do they behave".

PLM-CFG-03 scope. Wallet catalog admin: create, read, update, soft-delete, plus audit and lifecycle (DRAFT → ACTIVE → RETIRED). No workflow, no maker-checker, no Camunda. Pure catalog CRUD with DRL rules at every state transition. Wallet **types** are themselves a sub-catalog — admins seed and extend per deployment, identical pattern to `homepass_status_code` in RLM-CFG-01.

Out of scope for this DD (explicitly):

- **Per-customer wallet ledger / balances** — owned by BIL - 06 (future Wallet Operations DD). PLM-CFG-03 defines what wallets exist; BIL-06 maintains each customer's balance in each wallet, ledger entries, top-up history, expiry processing.
- **Accounts receivable (AR)** for postpaid subscriptions — owned by future invoicing / dunning DDs (existing sofXInvoicing + BIL - 04 dunning, generalized). AR is not a wallet; a customer who owes the operator money has open invoices, not a negative wallet.
- **Billing mode** (prepaid vs postpaid) — owned by the future Subscription Lifecycle DD. The Subscription entity carries the customer's chosen billing mode for each contract. PLM-CFG-03 declares per-wallet *applicability* across billing modes via the applicability field but does not own the mode itself.
- **Buckets / non-monetary allowances** — data MB, voice minutes, SMS counts. Deferred to a future DD if a deployment needs them. The wallet model here is strictly monetary-equivalent.

Downstream modules (BIL - 01 invoicing, BIL - 06 wallet operations, FUL - 03 fulfillment, CVM, search index) consume Wallet data via Kafka events and keep their own caches in sync. plm never calls back into them.

2. Business rules

These are the rules the system enforces on every Wallet operation. They must be agreed before any technical implementation. Each rule has an ID; failures return BUSINESS_RULE_ERROR with the rule ID in the response.

Wallet catalog rules

ID	Rule	Triggered on
R-PLM-CFG-03-W-1	Wallet code must be unique within the catalog. The code is the business identifier used by other modules (walletRef).	Create, Update
R-PLM-CFG-03-W-2	Wallet code is immutable once any Service, Package, or Discount references it. The code is the cross-module ref; changing it would break those references.	Update
R-PLM-CFG-03-W-3	A Wallet must reference an ACTIVE wallet_type catalog entry (see WalletType rules below) on Create and on every Update. A retired wallet type cannot be assigned to a new wallet.	Create, Update
R-PLM-CFG-03-W-4	A Wallet's currency must be a valid ISO 4217 code (KES, TZS, UGX, XOF, USD, ...). One wallet entry holds one currency only — this is a simplification chosen for finance reconciliation simplicity. A deployment offering multiple currencies seeds a wallet per (type, currency) pair (e.g., MONEY_KES, MONEY_TZS).	Create, Update
R-PLM-CFG-03-W-5	A Wallet's currency is immutable once any Service, Package, or Discount references it. Changing currency on a referenced wallet would invalidate downstream pricing.	Update
R-PLM-CFG-03-W-6	A Wallet's decimal_precision (number of decimal places used in the balance) must be in [0, 4]. Default: 2 for currency wallets, 0 for points wallets (points are typically whole numbers). Once set on a wallet referenced by downstream modules, immutable (R-PLM-CFG-03-W-5 sibling).	Create, Update
R-PLM-CFG-03-W-7	A Wallet's applicability declares which subscription billing modes can use it: PREPAID_ONLY (the wallet is consulted only when charging a prepaid subscription — typical for top-up wallets), POSTPAID_ONLY (consulted only for postpaid subscriptions — e.g., a corporate goodwill credit specific to B2B accounts), or ANY (consulted for any subscription — typical for deposit, loyalty points, service credit, hardware buyback credit). Default depends on the linked wallet_type (see the seed table below). Once set on a wallet referenced by downstream modules, immutable.	Create, Update

ID	Rule	Triggered on
R-PLM-CFG-03-W-8	A Wallet's balance is always non-negative . The wallet represents value the customer holds, drawn down at charging time and topped up by recharge, bonus, or refund. The system never lets a wallet go below zero — the charging path is responsible for rejecting or partially applying a charge that would do so. This is the explicit contract PLM-CFG-03 makes with downstream modules: every wallet balance is ≥ 0 .	(contract, not validated here)
R-PLM-CFG-03-W-9	A Wallet's expires flag controls whether unspent balance has an expiry date. Default false (money wallets, deposits) vs true (loyalty points, promotional credits). When true, an expiry_period_days is required (positive integer) — the customer's balance entries each carry their own expiry timestamps based on this period. Mutable post-create.	Create, Update
R-PLM-CFG-03-W-10	A Wallet's charging_precedence (integer, lower = applied first) controls the order in which multiple eligible wallets are charged against for the same line. Default: 100. Two wallets with the same precedence apply alphabetically by code (deterministic, but admins SHOULD avoid duplicates).	Create, Update
R-PLM-CFG-03-W-11	A Wallet's refillable flag controls whether the wallet accepts top-ups (true for money, bonus, deposit, points — anything that can be replenished; false for one-shot promotional credits that expire without refill). Default: true. Once set on a wallet referenced by downstream modules, immutable.	Create, Update
R-PLM-CFG-03-W-12	A Wallet cannot transition to RETIRED while any Service, Package, or Discount references it. The admin must first detach those references or retire the referencing entities.	Retire (ACTIVE → RETIRED)
R-PLM-CFG-03-W-13	A Wallet cannot be soft-deleted unless it is RETIRED. (And by R-W-12, retirement already requires no live references, so soft-delete is safe.)	Delete
R-PLM-CFG-03-W-14	Wallet status transitions: only DRAFT → ACTIVE, ACTIVE → RETIRED, RETIRED → (soft-delete). No backwards transitions. A retired wallet is permanently retired; if a deployment needs that wallet again, they create a new one with a fresh code.	Activate, Retire
R-PLM-CFG-03-W-15	A Wallet linked to a wallet_type whose unit = 'points' MUST declare a points_to_currency_rate (positive number) — the conversion rate applied at charging time to translate a points balance to a currency amount. For wallets whose linked wallet_type.unit = 'currency', points_to_currency_rate MUST be null. The rate is immutable once any Service/Package/Discount references the wallet, on the same grounds as R-W-5 (currency immutability) — a rate change would silently revalue every customer's points balance.	Create, Update

WalletType catalog rules

wallet_type is itself a configurable catalog (same pattern as homepass_status_code), seeded with common types per deployment, extensible. Each WalletType is referenced by Wallets via wallet_type_code (FK to wallet_type.code).

ID	Rule	Triggered on
R-PLM-CFG-03-WT-1	WalletType code must be unique within the deployment.	Create, Update
R-PLM-CFG-03-WT-2	WalletType code is immutable once any Wallet references it.	Update
R-PLM-CFG-03-WT-3	A WalletType cannot be soft-deleted while referenced by any Wallet (even retired wallets — historical refs must remain stable). Retire instead.	Delete

ID	Rule	Triggered on
R-PLM-CFG-03-WT-4	WalletType unit (free-text label for the value, typically currency or points) is informational; not enforced by code but visible in UIs and reports. The default seed uses currency for the monetary wallet types and points for LOYALTY_POINTS.	Create, Update

Full DRL implementation of these rules is in the "Rules (Drools)" section further down.

3. Module owner and impacted modules

Role	Module	What it does in this process
Owner	plm	Owns Wallet + WalletType data, REST APIs, the catalog admin UI, the DRL rules contribution
Impacted (downstream consumers)	billing (BIL-01 invoicing, BIL-06 wallet operations / ledger), plm (SIP-01 Package references, PLM-CFG-01 Service defaultWallet ref, PLM-CFG-04 Discount eligibleWalletRefs), search index, CVM	Consume WalletCreated/Updated/Activated/Retired and WalletTypeCreated/Updated/Retired Kafka events; refresh local caches
Gateways	—	None used in this process (pure CRUD)
Engines	Drools KIE	Rule evaluation on every create/update/state transition/delete

No workflow, no Camunda. No external system calls. No payment, no notification, no provisioning. PLM-CFG-03 is pure catalog admin.

4. Foundation references

DB → FOUNDATION_DB.md · Drools → FOUNDATION_DROOLS.md · Kafka → FOUNDATION_KAFKA.md · Cache → FOUNDATION_CACHE.md · Auth → FOUNDATION_AUTH.md.

Technical design

Module placement

Layer	Where
Frontend	BSS Backoffice UI (admin app) — new "Wallet Catalog" screens under Billing config
Backend	Existing plm module — new sub-area wallet-catalog/
Rules	Existing plm-rules KJAR — new DRL files under rules/wallet-catalog/
Events	Existing Kafka cluster — new topic family sophix.plm.wallet.* and sophix.plm.wallettype.*

Single-tenant: same codebase per country, config + rule packages loaded at startup. No deployment column.

plm module — ecosystem (PLM-CFG-03 view)

```

flowchart LR
    UI[Backoffice UI Wallet Catalog] -->|REST| PLM[plm module wallet-catalog]
    PLM -->|invoke| RE[Rules Engine]
    PLM -->|publish| EB[Kafka]
    EB --> BILINV[billing BIL-01 invoicing cache]
    EB --> BILWAL[billing BIL-06 wallet ledger cache]
    EB --> SVCCAT[plm Service catalog cache]
    EB --> PKGCAT[plm Package catalog cache]
    EB --> DISCAT[billing Discount catalog cache]
    EB --> SEARCH[search index]
    SVCCAT -.read walletRef.-> PLM
    PKGCAT -.read walletRef.-> PLM
    DISCAT -.read eligibleWalletRefs.-> PLM

```

plm/wallet-catalog never calls customer services, billing engine, or fulfillment workflows. It exposes catalog data via REST and emits Kafka events. Downstream consumers cache, never call back synchronously.

Codebase

PLM-CFG-03 adds to the existing plm module. Stack stays as chosen for plm (Java/Spring Boot OR Laravel — same contract).

Java/Spring Boot layout

```
modules/plm/
  src/main/java/com/sophix/plm/
    service-catalog/
      wallet-catalog/
        domain/
          Wallet.java
          WalletStatus.java
          WalletType.java
          WalletTypeStatus.java
        repository/
          WalletRepository.java
          WalletTypeRepository.java
        service/
          WalletService.java
          WalletValidationService.java
          WalletTypeService.java
          WalletTypeValidationService.java
        web/rest/
          WalletResource.java
          WalletTypeResource.java
          dto/...
        event/
          WalletEventPublisher.java
          WalletTypeEventPublisher.java
      shared/
        messaging/
          ...
    src/main/resources/
      config/liquibase/changelog/plm/
        ...
        050_wallet_type.xml
        051_wallet.xml
```

← PLM-CFG-01 (existing)
← PLM-CFG-03 (this DD)

← invokes Drools

← existing outbox/inbox workers

← existing PLM-CFG-01 changesets

Laravel layout (if plm is Laravel)

```
modules/plm/
  app/
    Domain/ServiceCatalog/
    Domain/WalletCatalog/
    Models/{Wallet, WalletType}.php
    Services/{WalletService, WalletValidationService, WalletTypeService, WalletTypeValidationService}.php
    Http/Controllers/{WalletController, WalletTypeController}.php
    Http/Requests/...
    Events/{WalletCreated, WalletUpdated, WalletActivated, WalletRetired, WalletTypeCreated, WalletTypeUpdated, WalletTy
  database/migrations/
    2026_05_13_000050_create_wallet_type_table.php
    2026_05_13_000051_create_wallet_table.php
```

DRL files live in the existing modules/plm-rules/ KJAR (see FOUNDATION_DR00LS.md). Laravel plm calls KIE Server over HTTPS REST.

Foundation-specific configurations

Foundation	Configuration for PLM-CFG-03
DB	Existing plm database in cluster A (Config & Catalog). New tables wallet_type, wallet in schema plm_catalog. Reuses existing messaging tables in shared messaging schema.
Drools	Adds DRL files to the existing plm-rules KJAR under rules/wallet-catalog/. Adds Fact classes for Wallet and WalletType.
Kafka	New topic families: sophix.plm.wallet.*, sophix.plm.wallettype.*. Key format: plm:wallet:{code}, plm:wallettype:{code}.
Cache	Redis cache for Wallet catalog reads (hit on every charge / invoice line): sophix:plm:wallet:{code} 24h TTL; admin updates invalidate via Kafka consumer.
Auth	Roles: WALLET_ADMIN (full CRUD on Wallet + WalletType), WALLET_VIEWER (read-only). JWT verified locally by plm.

Frontend

BSS Backoffice UI, Billing config section. Two sub-areas, each with list / detail / edit screens.

Screen	Purpose
Wallet Type — List	Browse the deployment's configured wallet types; filter by status, unit
Wallet Type — Detail / Edit	View and modify a WalletType; soft-delete; create new
Wallet — List	Browse wallets; filter by status, type, currency
Wallet — Detail / Edit	View and modify a Wallet; activate; retire; soft-delete; create new

Backoffice consumes the REST API directly. No custom DSL; the form fields map 1:1 to the rule columns.

Data model

wallet_type

Per-deployment configurable catalog of wallet type codes. Seeded at deployment provisioning with a sensible default set; extensible by admins.

Column	Type	Notes		
id	TEXT PK	ULID		
code	TEXT UNIQUE NOT NULL	short uppercase identifier (e.g. MONEY, BONUS, PROMO, DEPOSIT, LOYALTY_POINTS, SERVICE_CREDIT, BUYBACK_CREDIT)		
description	TEXT NOT NULL	human-readable label		
unit	TEXT NOT NULL	unit-of-value label, typically currency or points; informational, used by UIs and reports		
display_order	INTEGER NOT NULL DEFAULT 0	UI ordering		
status	TEXT NOT NULL	DRAFT \	ACTIVE \	RETIRED
created_at / updated_at / etc.		standard audit cols		
retired_at / deleted_at		standard soft-delete		

Indexes: idx_wallet_type_code UNIQUE on (code).

Recommended default seed (for new deployments)

Code	Description	Unit	Applicability default	Source semantics
MONEY	Customer top-up balance	currency	PREPAID_ONLY	Customer-funded (cash, card, mobile money, bank transfer); the "real money" balance. Postpaid customers do not top up — they are invoiced.
BONUS	Operator-granted credit	currency	PREPAID_ONLY	Goodwill, retention — operator-funded credit consumed at prepaid charging. Postpaid analogues are invoice-line credits, not wallet credits.
PROMO	Promotional credit	currency	PREPAID_ONLY	One-shot, time-limited — welcome bonus, referral reward, campaign credit. Typically charged first because of expiry.
DEPOSIT	Refundable deposit	currency	ANY	Held at install against contract termination or equipment damage. Operator-controlled; cannot be spent by the customer; finance liability. Applies to both prepaid and postpaid subscriptions.

Code	Description	Unit	Applicability default	Source semantics
LOYALTY_POINTS	Loyalty points	points	ANY	Activity-earned (longevity, payments, referrals); convertible to currency at the wallet's configured <code>points_to_currency_rate</code> . Redeemable by prepaid customers against their next cycle charge, by postpaid customers against their next invoice.
SERVICE_CREDIT	SLA-driven compensation	currency	ANY	System-issued on SLA breach. Applied to next prepaid charge or next postpaid invoice depending on the subscription's billing mode.
BUYBACK_CREDIT	Hardware-buyback credit	currency	ANY	Issued when a customer returns equipment in good condition at contract end; redeemable against future purchases (prepaid charge or postpaid invoice) or refundable per deployment policy.

All defaults are monetary-equivalent. The unit is currency in all cases except LOYALTY_POINTS, where the unit is points and a per-wallet conversion rate (R-W-15) translates points to currency at charging time.

The applicability defaults reflect a natural separation: **top-up wallets** (MONEY, BONUS, PROMO) are inherent to the prepaid model — a postpaid customer doesn't top up because the operator extends credit instead. **Ancillary wallets** (DEPOSIT, LOYALTY_POINTS, SERVICE_CREDIT, BUYBACK_CREDIT) apply equally to both billing modes — a customer's deposit is held regardless of mode; loyalty points are earned by anyone; SLA service credit is issued to anyone; buyback credit is issued to anyone returning equipment.

A deployment may seed fewer types (e.g. only MONEY and DEPOSIT for an operator that doesn't run promos or loyalty), or add types unique to its product mix. Common future additions:

- WHOLESALE (typically PREPAID_ONLY) — for B2B reseller prepaid balances if Wananchi enters the wholesale market
- A postpaid-specific goodwill credit (typically POSTPAID_ONLY) — for operators that want to issue corporate goodwill that should never affect prepaid charging
- Other operator-specific credit categories (e.g., EARLY_TERMINATION_FEE_WAIVER, REFERRAL_BONUS split from generic PROMO)

The catalog is admin-managed; new types and applicability rules do not require a code release.

wallet

The deployment's catalog of wallets. Each wallet entry is referenced by Service defs (`defaultWallet` from PLM-CFG-01), Discount defs (`eligibleWalletRefs` from PLM-CFG-04 once added), Package defs (where applicable), and is the addressing key for the customer's per-wallet balance ledger maintained by BIL-06.

Column	Type	Notes		
id	TEXT PK	ULID		
code	TEXT UNIQUE NOT NULL	short business identifier — MONEY_KES, BONUS_KES, PROMO_KES, DEPOSIT_KES, LOYALTY_POINTS_KES, ... — used as <code>walletRef</code> from other catalogs		
description	TEXT NOT NULL	human-readable label		
wallet_type_code	TEXT NOT NULL	FK to <code>wallet_type.code</code> (R-W-3)		
currency	TEXT NOT NULL	ISO 4217 code (R-W-4); one wallet = one currency		
decimal_precision	INTEGER NOT NULL DEFAULT 2	0 for points wallets, 2 for currency wallets, up to 4 (R-W-6)		

Column	Type	Notes		
applicability	TEXT NOT NULL DEFAULT 'ANY'	PREPAID_ONLY \	POSTPAID_ONLY \	ANY (R-W-7); declares which subscription billing modes can use this wallet at charging time
expires	BOOLEAN NOT NULL DEFAULT false	per R-W-9		
expiry_period_days	INTEGER NULL	required when expires = true; positive integer (R-W-9)		
charging_precedence	INTEGER NOT NULL DEFAULT 100	lower applies first (R-W-10)		
refillable	BOOLEAN NOT NULL DEFAULT true	per R-W-11		
points_to_currency_rate	NUMERIC(18, 6) NULL	required when the linked wallet_type.unit = 'points' (per R-W-15); the rate that converts 1 point to the wallet's currency at charging time. Example: rate 0.01 means 100 points = 1 KES. NULL for currency-unit wallets.		
status	TEXT NOT NULL	DRAFT \	ACTIVE \	RETIRED
created_at / updated_at / etc.		standard audit cols		
retired_at / deleted_at		standard soft-delete		

Indexes:

- idx_wallet_code UNIQUE on (code)
- idx_wallet_type on (wallet_type_code) — supports reporting and "all wallets of this type" lookups
- idx_wallet_status partial on (status) WHERE status = 'ACTIVE' — supports the hot read from BIL-01 / BIL-06 at charging time
- idx_wallet_currency on (currency) — supports reporting by currency

audit_log_entry (shared)

Existing pattern per HLD §13. Two new entity types: Wallet, WalletType. Each operation logged.

Messaging tables

Existing outbox/inbox tables in shared messaging schema. New event publisher classes for Wallet and WalletType.

Rules (Drools)

Adds DRL files to the existing plm-rules KJAR (no new KJAR). Files placed under rules/wallet-catalog/.

DRL files

```

modules/plm-rules/                                ← EXISTING KJAR
  src/main/resources/
    rules/
      service-catalog/                             ← PLM-CFG-01 (existing)
      wallet-catalog/                               ← PLM-CFG-03 (this DD)
        wallet-validation-create-update.drl
        wallet-validation-activate.drl
        wallet-validation-retire.drl
        wallet-validation-delete.drl
        wallettype-validation-create-update.drl
        wallettype-validation-delete.drl

```

The plm-rules kmodule.xml adds the new packages:

```

<kmodule xmlns="http://www.drools.org/xsd/kmodule">
  <kbase name="plm" packages="rules.service-catalog,
                             rules.wallet-catalog">
    <ksession name="plmSession" type="stateless"/>
  </kbase>
</kmodule>

```

Fact classes

```
@Data
@NoArgsConstructor
@AllArgsConstructor
public class WalletFact {
    private String id;
    private String code;
    private String description;
    private String walletTypeCode;
    private String currency;
    private int decimalPrecision;
    private boolean expires;
    private Integer expiryPeriodDays;           // null if expires=false
    private int chargingPrecedence;
    private boolean refillable;
    private String applicability;               // PREPAID_ONLY | POSTPAID_ONLY | ANY
    private BigDecimal pointsToCurrencyRate;    // non-null only for points-unit wallets
    private String status;                     // DRAFT | ACTIVE | RETIRED
}

@Data
@NoArgsConstructor
@AllArgsConstructor
public class WalletTypeFact {
    private String code;
    private String description;
    private String unit;
    private String status;                     // DRAFT | ACTIVE | RETIRED
}

@Data
@NoArgsConstructor
@AllArgsConstructor
public class WalletTypeExistsFact {
    private String code;
    private boolean exists;
    private boolean active;                   // true if status=ACTIVE
}

@Data
@NoArgsConstructor
@AllArgsConstructor
public class WalletTypeUnitFact {
    private String code;
    private String unit;                     // 'currency' | 'points' | ...
}

@Data
@NoArgsConstructor
@AllArgsConstructor
public class WalletReferencedFact {
    private String walletCode;
    private boolean referencedByService;      // true if any plm.service.default_wallet = walletCode
    private boolean referencedByPackage;
    private boolean referencedByDiscount;     // true if any billing.discount.eligible_wallets contains walletCode
    public boolean isReferenced() {
        return referencedByService || referencedByPackage || referencedByDiscount;
    }
}

@Data
@NoArgsConstructor
@AllArgsConstructor
public class WalletTypeReferencedFact {
    private String code;
    private boolean referenced;               // true if any wallet.wallet_type_code = code
}

@Data
@NoArgsConstructor
@AllArgsConstructor
public class ValidationOutcome {
    private boolean valid;
    private List<String> errors;
    private List<String> warnings;
}
```

wallet-validation-create-update.drl

```
package rules.wallet-catalog;

import com.sophix.plm.facts.wallet.WalletFact;
```

```

import com.sophix.plm.facts.wallet.WalletTypeExistsFact;
import com.sophix.plm.facts.wallet.ValidationOutcome;

rule "R-PLM-CFG-03-W-3: wallet type must exist and be ACTIVE"
when
    $w : WalletFact()
    $wt : WalletTypeExistsFact(code == $w.walletTypeCode,
                               (exists == false || active == false))
    $out : ValidationOutcome()
then
    if (!$wt.isExists()) {
        $out.getErrors().add("R-PLM-CFG-03-W-3: wallet type '" + $w.getWalletTypeCode() + "' does not exist");
    } else {
        $out.getErrors().add("R-PLM-CFG-03-W-3: wallet type '" + $w.getWalletTypeCode() + "' is not ACTIVE");
    }
    modify($out) { setValid(false); }
end

rule "R-PLM-CFG-03-W-4: currency must be a valid ISO 4217 code"
when
    $w : WalletFact()
    $out : ValidationOutcome()
    eval(!IsoCurrencies.isValid($w.getCurrency()))
then
    $out.getErrors().add("R-PLM-CFG-03-W-4: '" + $w.getCurrency() + "' is not a valid ISO 4217 currency code");
    modify($out) { setValid(false); }
end

rule "R-PLM-CFG-03-W-6: decimal_precision must be in [0, 4]"
when
    $w : WalletFact(decimalPrecision < 0 || decimalPrecision > 4)
    $out : ValidationOutcome()
then
    $out.getErrors().add("R-PLM-CFG-03-W-6: decimal_precision " + $w.getDecimalPrecision() + " out of range [0, 4]");
    modify($out) { setValid(false); }
end

rule "R-PLM-CFG-03-W-7: applicability must be PREPAID_ONLY | POSTPAID_ONLY | ANY"
when
    $w : WalletFact()
    eval($w.getApplicability() == null
         || !java.util.Arrays.asList("PREPAID_ONLY", "POSTPAID_ONLY", "ANY")
            .contains($w.getApplicability()))
    $out : ValidationOutcome()
then
    $out.getErrors().add("R-PLM-CFG-03-W-7: applicability must be PREPAID_ONLY, POSTPAID_ONLY, or ANY");
    modify($out) { setValid(false); }
end

rule "R-PLM-CFG-03-W-9: expiry_period_days required when expires = true"
when
    $w : WalletFact(expires == true,
                    (expiryPeriodDays == null || expiryPeriodDays <= 0))
    $out : ValidationOutcome()
then
    $out.getErrors().add("R-PLM-CFG-03-W-9: expiry_period_days must be a positive integer when expires=true");
    modify($out) { setValid(false); }
end

rule "R-PLM-CFG-03-W-15a: points_to_currency_rate required for points wallets"
when
    $w : WalletFact()
    $wtu : WalletTypeUnitFact(code == $w.walletTypeCode, unit == "points")
    eval($w.getPointsToCurrencyRate() == null
         || $w.getPointsToCurrencyRate().signum() <= 0)
    $out : ValidationOutcome()
then
    $out.getErrors().add("R-PLM-CFG-03-W-15: points_to_currency_rate (positive) required when wallet_type.unit='points'");
    modify($out) { setValid(false); }
end

rule "R-PLM-CFG-03-W-15b: points_to_currency_rate must be null for currency wallets"
when
    $w : WalletFact(pointsToCurrencyRate != null)
    $wtu : WalletTypeUnitFact(code == $w.walletTypeCode, unit == "currency")
    $out : ValidationOutcome()
then
    $out.getErrors().add("R-PLM-CFG-03-W-15: points_to_currency_rate must be null when wallet_type.unit='currency'");
    modify($out) { setValid(false); }
end

```

R-W-2 (code immutable on referenced wallets) and R-W-5 (currency immutable on referenced wallets) are enforced at the service layer by comparing incoming fields against the stored row; DRL is invoked only on the catalog-side validations. R-W-1 (code unique within the catalog) is enforced as a DB unique index plus a friendly DRL message wrapper.

wallet-validation-retire.drl

```
package rules.wallet-catalog;

import com.sophix.plm.facts.wallet.WalletFact;
import com.sophix.plm.facts.wallet.WalletReferencedFact;
import com.sophix.plm.facts.wallet.ValidationOutcome;

rule "R-PLM-CFG-03-W-12: cannot retire wallet while referenced"
when
    $w : WalletFact()
    $r : WalletReferencedFact(walletCode == $w.code, referenced == true)
    $out : ValidationOutcome()
then
    StringBuilder refs = new StringBuilder();
    if ($r.isReferencedByService()) refs.append("Service ");
    if ($r.isReferencedByPackage()) refs.append("Package ");
    if ($r.isReferencedByDiscount()) refs.append("Discount ");
    $out.getErrors().add("R-PLM-CFG-03-W-12: cannot retire wallet '" + $w.getCode() + "' - referenced by: " + refs);
    modify($out) { setValid(false); }
end
```

R-W-13 (cannot delete unless RETIRED) is a service-layer check before invoking any DRL (it's a status assertion, not a fact-based rule).

wallettype-validation-create-update.drl

```
package rules.wallet-catalog;

import com.sophix.plm.facts.wallet.WalletTypeFact;
import com.sophix.plm.facts.wallet.ValidationOutcome;

rule "R-PLM-CFG-03-WT-4: unit is required and non-empty"
when
    $wt : WalletTypeFact(unit == null || unit.trim().length() == 0)
    $out : ValidationOutcome()
then
    $out.getErrors().add("R-PLM-CFG-03-WT-4: unit is required");
    modify($out) { setValid(false); }
end
```

R-WT-1 (code unique) and R-WT-2 (code immutable) are enforced at the service / DB layer. R-WT-3 (cannot delete while referenced) is in wallettype-validation-delete.drl:

```
rule "R-PLM-CFG-03-WT-3: cannot delete wallet type while referenced"
when
    $wt : WalletTypeFact()
    $r : WalletTypeReferencedFact(code == $wt.code, referenced == true)
    $out : ValidationOutcome()
then
    $out.getErrors().add("R-PLM-CFG-03-WT-3: cannot delete wallet type '" + $wt.getCode() + "' - referenced by existing");
    modify($out) { setValid(false); }
end
```

Events (Kafka)

Standard envelope (per FOUNDATION_KAFKA.md). New topics:

- sophix.plm.wallet.created
- sophix.plm.wallet.updated
- sophix.plm.wallet.activated
- sophix.plm.wallet.retired
- sophix.plm.wallettype.created
- sophix.plm.wallettype.updated
- sophix.plm.wallettype.retired

sophix.plm.wallet.created example payload

```
{
  "eventType": "WalletCreated",
  "aggregateId": "wlt_01HX...",
}
```

```

    "timestamp": "2026-05-13T10:00:00Z",
    "actor": "admin@wananchi.co.ke",
    "correlationId": "corr_01HX...",
    "payload": {
      "code": "MONEY_KES",
      "description": "Money credit, KES",
      "walletTypeCode": "MONEY",
      "currency": "KES",
      "decimalPrecision": 2,
      "expires": false,
      "expiryPeriodDays": null,
      "chargingPrecedence": 100,
      "refillable": true,
      "applicability": "PREPAID_ONLY",
      "pointsToCurrencyRate": null,
      "status": "DRAFT"
    }
  }
}

```

sophix.plm.wallet.activated

Emitted when a Wallet transitions DRAFT → ACTIVE. Carries the full wallet snapshot so consumers can refresh their caches without a round-trip.

sophix.plm.wallet.retired

Emitted when ACTIVE → RETIRED. Downstream consumers (BIL-01, BIL-06) must not allow new charges against this wallet from this point.

sophix.plm.wallettype.*

Same envelope shape with WalletType payloads.

Exceptions

Status	Code	Meaning
400	BUSINESS_RULE_ERROR	One or more DRL rules failed; response body lists failed rule IDs
400	VALIDATION_ERROR	Field-level validation failed (missing required, bad type)
404	NOT_FOUND	Wallet or WalletType not found
409	CONFLICT	Soft-delete blocked by references (R-W-12 / R-WT-3) or lifecycle violation (R-W-14)
422	IMMUTABLE_FIELD	Attempt to modify an immutable field (code, currency, decimal_precision, applicability, refillable on a referenced wallet — R-W-2, W-5, W-6, W-7, W-11)

APIs (in plm/wallet-catalog/)

Wallet endpoints (CRUD + lifecycle)

- GET /api/wallets — list; query params: status, walletTypeCode, currency, page/size
- GET /api/wallets/{id} — read one
- GET /api/wallets/by-code/{code} — read by business code (the lookup used by Service/Package/Discount cross-refs)
- POST /api/wallets — create (DRAFT). Body: {code, description, walletTypeCode, currency, decimalPrecision?, applicability?, expires?, expiryPeriodDays?, chargingPrecedence?, refillable?, pointsToCurrencyRate?}. DRL validation; returns 400 if rules fail.
- PATCH /api/wallets/{id} — update; immutable fields (R-W-2, W-5, W-6, W-7, W-11) cannot be changed once referenced (service-layer check).
- POST /api/wallets/{id}/activate — DRAFT → ACTIVE. DRL re-validates create-rules (defensive).
- POST /api/wallets/{id}/retire — ACTIVE → RETIRED. R-W-12 enforced.
- DELETE /api/wallets/{id} — soft-delete. R-W-13 enforced (must be RETIRED).

WalletType endpoints (CRUD + lifecycle)

- GET /api/wallet-types — list; query params: status, page/size
- GET /api/wallet-types/{id} — read one
- GET /api/wallet-types/by-code/{code} — read by business code
- POST /api/wallet-types — create (DRAFT). Body: {code, description, unit, displayOrder?}.
- PATCH /api/wallet-types/{id} — update; code immutable once referenced (R-WT-2).
- POST /api/wallet-types/{id}/activate — DRAFT → ACTIVE
- POST /api/wallet-types/{id}/retire
- DELETE /api/wallet-types/{id} — soft-delete (R-WT-3 enforced)

Example: create a money wallet

Request:

```
POST /api/wallets
{
  "code": "MONEY_KES",
  "description": "Money credit (KES)",
  "walletTypeCode": "MONEY",
  "currency": "KES",
  "decimalPrecision": 2,
  "applicability": "PREPAID_ONLY",
  "refillable": true,
  "chargingPrecedence": 100
}
```

Response 201:

```
{
  "id": "wlt_01HX...",
  "code": "MONEY_KES",
  "status": "DRAFT",
  "description": "Money credit (KES)",
  "walletTypeCode": "MONEY",
  "currency": "KES",
  "decimalPrecision": 2,
  "applicability": "PREPAID_ONLY",
  "expires": false,
  "expiryPeriodDays": null,
  "chargingPrecedence": 100,
  "refillable": true,
  "pointsToCurrencyRate": null,
  "createdAt": "2026-05-13T10:00:00Z"
}
```

Example: create a deposit wallet (applicable to both billing modes)

Request:

```
POST /api/wallets
{
  "code": "DEPOSIT_KES",
  "description": "Customer deposit (KES)",
  "walletTypeCode": "DEPOSIT",
  "currency": "KES",
  "decimalPrecision": 2,
  "applicability": "ANY",
  "refillable": true,
  "chargingPrecedence": 200
}
```

A customer's deposit is held against contract termination or equipment damage regardless of whether the subscription is prepaid or postpaid — hence applicability: ANY. The wallet balance is never negative; if damage costs exceed the deposit, the excess is handled outside this module (it becomes part of the postpaid invoicing path or the prepaid charging path per the subscription's billing mode).

Example: create a loyalty-points wallet (with conversion rate)

Request:

```
POST /api/wallets
{
  "code": "LOYALTY_POINTS_KES",
  "description": "Loyalty points convertible to KES",
  "walletTypeCode": "LOYALTY_POINTS",
  "currency": "KES",
  "conversionRate": 100
}
```

```

    "decimalPrecision": 0,
    "applicability": "ANY",
    "expires": true,
    "expiryPeriodDays": 365,
    "refillable": true,
    "chargingPrecedence": 50,
    "pointsToCurrencyRate": 0.01
}

```

Interpretation: 100 points = 1 KES at redemption. Points balance is whole-numbered (decimalPrecision: 0); the converted currency amount inherits the operator's KES precision at charging time. The wallet has lower chargingPrecedence than money (50 vs 100), so points are spent before money — encouraging redemption before they expire (365 days). applicability: ANY because loyalty points are earned by any customer regardless of billing mode.

Sequence — create wallet

```

sequenceDiagram
    participant U as Wallet Admin
    participant UI as Backoffice UI
    participant PLM as plm module
    participant KIE as KIE Server
    participant DB as plm DB
    participant K as Kafka

    U->>UI: fill wallet form
    UI->>PLM: POST create wallet
    PLM->>PLM: load WalletType for the requested code
    PLM->>KIE: evaluate create rules
    KIE-->>PLM: ok
    PLM->>DB: INSERT wallet
    PLM->>DB: INSERT outbox WalletCreated
    PLM->>K: outbox worker publishes WalletCreated
    PLM-->>UI: 201 Created

```

Sequence — retire wallet

```

sequenceDiagram
    participant U as Wallet Admin
    participant PLM as plm module
    participant SVC as plm service-catalog
    participant PKG as plm package-catalog
    participant DIS as billing discount-catalog
    participant KIE as KIE Server
    participant DB as plm DB
    participant K as Kafka

    U->>PLM: POST retire wallet
    PLM->>SVC: GET services referencing this wallet
    PLM->>PKG: GET packages referencing this wallet
    PLM->>DIS: GET discounts referencing this wallet
    PLM->>KIE: evaluate R-W-12 with WalletReferencedFact
    KIE-->>PLM: ok or blocked
    alt allowed
        PLM->>DB: UPDATE wallet status=RETIRED
        PLM->>DB: INSERT outbox WalletRetired
        PLM->>K: WalletRetired event
        PLM-->>U: 200 OK
    else blocked
        PLM-->>U: 409 Conflict with reference list
    end

```

Dependencies

- **PLM-CFG-01** (Service Catalog) — exposes defaultwallet field on Service def referencing a Wallet code from this DD. Already in place (R-PLM-CFG-01-2). No change needed in PLM-CFG-01 from this DD.
- **PLM-CFG-04** (Discount Catalog) — exposes eligiblewalletRefs for WALLET_CREDIT discount type. The DD's dangling reference to "future Wallet DD" is now resolved by PLM-CFG-03. PLM-CFG-04 will be updated to depend on this DD rather than holding the reference open.
- **SIP-01** (Package Management) — references Wallet codes where Packages declare wallet associations (e.g., a Package contributes value to a specific wallet on purchase).
- **BIL-01** (future Invoicing) — reads wallet catalog at invoice generation time; charges against the customer's per-wallet balance maintained by BIL-06.
- **BIL-06** (future Wallet Operations) — maintains the per-customer wallet ledger (top-ups, charges, expiries, history). PLM-CFG-03 owns the wallet *catalog*; BIL-06 owns the per-customer *balance*. The split is the standard PLM-vs-BIL separation: PLM defines what wallets exist and how they behave; BIL applies those rules to per-customer operations.

B — Current TZ

TZ Sophix has **no Wallet entity at all**. The TZ system is fundamentally **postpaid**: it issues PRO FORMA invoices at the cycle close (per BIL - 01 / QuartzGenerateCycle), runs Quartz-driven dunning waves (BIL - 03 / BIL - 04) for unpaid invoices, and ultimately suspends customers via AAA when dunning escalates. No prepaid funding, no wallet ledger, no balance-based suspension.

The wallet concept is not in the TZ codebase. Sources:

- The **Sophix V3 Format GAP** analysis (G - 04 - 1) explicitly classifies the wallet module as an **XL net-new effort**: *"Module structurant et nouveau. Nécessite un nouveau microservice (sofxWallet) ou une extension lourde de sofxInvoicing. Gère le ledger source-of-truth, la concurrence, les buckets, l'audit trail."* 8 sprints estimated, priority P1.
- The same GAP analysis states the structural gap as: *"Le modèle prépayé wallet TVD de Wananchi (auto-debit, auto-settle, pas de dunning postpaid) est radicalement différent du modèle Sophix TZ qui est principalement postpaid (PRO FORMA invoice, dunning B2C/B2B avec waves Quartz, suspension AAA). C'est l'écart structurant qui définit la majeure partie de l'effort."*
- The wallet taxonomy (TVD for Internet/TV, Phone for Voice) comes from **Broadhub (BH) / Wananchi Kenya**, not from TZ. It's documented in the BH business-process module (APPFREESN - Module_Billing) and the eTOM business-process review for Wananchi, both describing the **target** environment.

So the V3 effort is not "modify the TZ wallet" — it's "introduce a wallet catalog and engine that doesn't yet exist in the codebase, drawing the model from Wananchi's existing Broadhub patterns and generalizing it for multi-deployment use".

What does exist in TZ Sophix today that is adjacent:

- sofxInvoicing owns the postpaid invoice lifecycle: PRO FORMA generation, payment intake, dunning. Reusable as the postpaid path going forward (separate DDs).
- PaymentResource with callback handling (Mixx by Yas and generic webhooks) for inbound payments — reusable for the prepaid recharge intake once it's repurposed to credit a wallet instead of settle an invoice (per GAP G - 05 - 1).
- **Cash Drawer module** for shop / cash payments — reusable.
- **No wallet entity, no wallet ledger, no recharge-to-balance flow, no auto-settle-from-wallet flow.**

The Wananchi target is documented in the **Broadhub** materials:

- Two operational wallets per customer in BH/KE today: TVD (Internet + TV settlement at cycle close) and Phone (Voice usage-based debit). The Voice wallet drains as the VoIP switch rates calls; the Internet/TV wallet drains at bill cycle close.
- Refund mechanics via "ADJ" (adjustment) entries on the customer ledger — credits posted with mandatory notes, thresholds requiring approval. The Wananchi GAP documents a negative wallet ADJ avec note obligatoire pattern for refund debit-notes (BP - 06), but this is a **ledger entry sign convention** (credit positive, debit-note negative), NOT a "wallet that can go below zero". The wallet balance itself is always non-negative; debit-notes net out previously-posted credits and the wallet is recomputed.
- No applicability distinction in BH today — wallets are implicitly prepaid because BH/Wananchi operates a pure-prepaid model. V3 generalizes by adding applicability so the same catalog can serve future deployments that also need postpaid (e.g., a future B2B segment with corporate billing terms).

V3 builds the wallet catalog from scratch as an extension of the plm module, generalizing the BH patterns and supporting both billing modes via the applicability field. Postpaid AR / dunning / invoicing concerns are handled by separate DDs (existing TZ sofxInvoicing + BIL - 04 dunning, generalized as needed) — not by this DD.

C — Gap & bridge

Changes from TZ → V3

Topic	TZ Sophix	V3 (this DD)
Wallet model	No wallet entity (postpaid model)	First-class catalog entity with full lifecycle (net-new module per GAP G - 04 - 1)

Topic	TZ Sophix	V3 (this DD)
Wallet types	N/A	Configurable wallet_type catalog, seeded with 7 types (MONEY, BONUS, PROMO, DEPOSIT, LOYALTY_POINTS, SERVICE_CREDIT, BUYBACK_CREDIT)
Currency	Deployment-hardcoded	Per-wallet ISO 4217 field; multi-currency deployments seed one wallet per (type, currency) pair
Applicability across billing modes	N/A (TZ is postpaid only)	applicability per wallet (PREPAID_ONLY / POSTPAID_ONLY / ANY). Top-up wallets default to PREPAID_ONLY; ancillary wallets (deposit, loyalty, service credit, buyback) default to ANY so they work for postpaid customers too
Negative balance	(TZ has no wallet)	Not supported. Wallet balance is always ≥ 0 . The BH "negative wallet ADJ" pattern is a debit-note ledger entry, not a negative wallet; the resulting wallet balance is non-negative. Postpaid AR is a separate concern owned by future invoicing / dunning DDs
Expiry	N/A	Per-wallet flag with configurable period (R-W-9)
Charging precedence	N/A	Per-wallet integer; admins reorder (R-W-10)
Refillability	N/A	Per-wallet flag (R-W-11)
Cross-module refs	N/A	Validated walletRef codes; immutable once referenced
Lifecycle	N/A	DRAFT → ACTIVE → RETIRED with rules per transition
Events	N/A	Standard Kafka envelope per state change

Migration approach

TZ Sophix Services carry free-text wallet values that conflated allowance-buckets and money pools. V3 separates the two concerns: this DD handles only the monetary wallets; non-monetary allowances (data MB, voice minutes, SMS count) are deferred to a future Bucket DD. V3 migration for the wallet half:

1. **Seed** wallet_type with the default 7 monetary types (MONEY, BONUS, PROMO, DEPOSIT, LOYALTY_POINTS, SERVICE_CREDIT, BUYBACK_CREDIT).
2. **Audit TZ wallet field values** across Service defs. Classify each value as either (a) a monetary-equivalent pool that maps to a V3 wallet, or (b) an allowance bucket deferred to the future Bucket DD. Common monetary mappings: TZ free-text balance/credit fields → V3 MONEY_<currency>, deposit fields → V3 DEPOSIT_<currency>.
3. **Seed** wallet with one entry per (monetary type, currency) pair the deployment needs (e.g., MONEY_KES, BONUS_KES, PROMO_KES, DEPOSIT_KES, LOYALTY_POINTS_KES for a single-currency KE deployment).
4. **Rewrite Service defaultwallet** values from the free-text strings to the new wallet code references (handled in PLM-CFG-01's migration, not here).
5. **Set sensible defaults** on each wallet: MONEY / BONUS / PROMO get applicability = PREPAID_ONLY and refillable = true. PROMO gets expires = true with a deployment-specific period and lower precedence (e.g. 50) so it's spent first. DEPOSIT / LOYALTY_POINTS / SERVICE_CREDIT / BUYBACK_CREDIT get applicability = ANY. LOYALTY_POINTS gets pointsToCurrencyRate per the loyalty programme design. No wallet has a negative-balance configuration — wallet balances are always non-negative.
6. **Per-currency wallet split**: a deployment offering multiple currencies seeds one wallet per (type, currency) pair (MONEY_KES, MONEY_TZS, ...). The wallet code carries the currency suffix by convention.

The migration runs once at V3 cutover; thereafter, the catalog is admin-managed.

Dependencies

Dependency	Owner	Status
Service catalog (PLM-CFG-01) defaultwallet field references this catalog	plm team	✓ in place
Discount catalog (PLM-CFG-04) eligiblewalletRefs references this catalog	plm / billing team	✓ in place; the pending-DD note is now resolved

Dependency	Owner	Status
Wallet ledger (BIL-06) reads this catalog at charging time	billing team	■ pending (future DD)
Invoicing (BIL-01) reads wallet config to apply per-wallet rules	billing team	■ pending (future DD)

What's NEW in V3 / not in TZ Sophix

1. **First-class wallet catalog** with a full DRAFT → ACTIVE → RETIRED lifecycle, replacing TZ's implicit string field.
2. **Configurable wallet_type sub-catalog** — admins seed and extend; no code release to add new types.
3. **Per-wallet ISO 4217 currency** — multi-currency deployments seed one wallet per (type, currency) pair.
4. **Per-wallet operational rules** (applicability, expires, expiry_period_days, refillable, charging_precedence, points_to_currency_rate) — replaces hardcoded billing logic with configurable catalog data. Wallet balance is always non-negative by design; the V3 target BSS does NOT model wallets that can go below zero.
5. **Code immutability once referenced** — Services / Packages / Discounts cross-reference by walletRef; changing a wallet code would break those refs, so once referenced, code is locked.
6. **Standard Kafka events** for every catalog state change — downstream caches stay coherent without coupling.
7. **Rule-based validation** — every catalog operation runs DRL rules; failed rules return structured BUSINESS_RULE_ERROR responses with rule IDs.
8. **Wallet-only model** (no Bucket entity) — V3 V3 keeps it simple: the wallet has a single monetary balance used at charging time. If a future deployment needs the Bucket allocation pattern (free vs purchased vs promotional within one wallet), a separate DD adds it without changing this one.